# AI/ML Capstone Project

## End-to-End Data Engineering, Machine Learning, and AI Support Assistant

| | |
|---|---|
| Project Type | End-to-End AI/ML Application |
| Technology Stack | Python, Web Scraping, SQLite, Pandas, SQL, Scikit-learn, Streamlit, FastAPI, Uvicorn, ChromaDB, LangGraph, Docker |
| Institution | Dhanekula Institute of Engineering & Technology |
| Department | Computer Science and Engineering |
| Document | Detailed Technical Documentation |

> This document describes the project as recorded in its observed structure and test results. Items marked *to be verified* must be confirmed against the source files before formal submission.

# Table of Contents

**Table of Contents** 2

**1. Project Overview** 4

**2. Problem Statement** 4

**3. Project Objectives** 4

**4. System Architecture** 6

4.1 Data Engineering Architecture . . . 6

4.2 Analytics and Machine Learning Architecture . . . 7

4.3 AI Support Assistant Architecture . . . 8

**5. Project Directory Structure** 9

**6. Module One: Web Scraping and Data Engineering** 10

6.1 Introduction to Web Scraping . . . 10

6.2 Extracting Data from Web Pages . . . 10

6.3 Raw Data Storage . . . 10

6.4 Data Cleaning and Transformation . . . 10

6.5 SQLite Database Integration . . . 11

6.6 SQL Query Execution . . . 11

6.7 Pandas Validation . . . 11

6.8 Module Outputs . . . 11

**7. Module Two: Data Analytics and Machine Learning** 12

7.1 Overview . . . 12

7.2 Data Loading and Inspection . . . 12

7.3 Data Cleaning . . . 12

7.4 Exploratory Data Analysis . . . 12

7.5 Classification: Titanic Survival Prediction . . . 12

7.6 Random Forest Hyperparameter Tuning . . . 13

7.7 Model Serialisation . . . 13

7.8 Fare Regression . . . 13

7.9 Interactive Analytics Interface . . . 14

**8. Module Three: AI Support Assistant** 15

8.1 Purpose . . . 15

8.2 Policy Documents . . . 15

8.3 Document Ingestion . . . 15

8.4 Embeddings and Semantic Search . . . 15
8.5 LangGraph Workflow . . . 16
8.6 Prompt Engineering . . . 16
8.7 Request and Response Schemas . . . 16
8.8 FastAPI Backend . . . 16
8.9 Uvicorn and the API Server . . . 17
8.10 Streamlit-to-FastAPI Communication . . . 17
8.11 Testing the RAG Assistant . . . 18

**9. Module Four: Streamlit User Interface 19**

9.1 Overview . . . 19
9.2 Main Dashboard . . . 19
9.3 Analytics and ML Page . . . 19
9.4 AI Support Assistant Page . . . 19
9.5 Styling and Navigation . . . 19

**10. Running the Complete Application 19**

10.1 Start the FastAPI Backend . . . 19
10.2 Start the Streamlit Frontend . . . 20
10.3 Why Two Terminals Are Needed . . . 20

**11. Database Architecture: SQLite and ChromaDB 20**

**12. Error Handling and Troubleshooting 21**

12.1 FastAPI Backend Does Not Start . . . 21
12.2 Streamlit Cannot Connect to the Assistant . . . 21
12.3 Chatbot Returns Generic or Irrelevant Answers . . . 21
12.4 Model or Dataset Loading Failure . . . 21

**13. Technologies Used 22**

**14. Project Outcomes 22**

**15. Limitations and Future Enhancements 22**

15.1 Current Limitations . . . 22
15.2 Future Enhancements . . . 23

**16. Conclusion 23**

**Appendix A: Important Project Files 24**

**Appendix B: Demonstration Sequence 24**

# 1. Project Overview

The AI/ML Capstone Project is an integrated application that demonstrates multiple stages of a practical data and artificial intelligence workflow. It combines web scraping, data processing, relational database operations, exploratory data analysis, machine learning, retrieval-augmented generation (RAG), API development, and interactive web application development.

The project is organised into three major functional modules, unified by a Streamlit front end:

| Module | Description |
|---|---|
| Data Engineering Pipeline | Collects book information from a website, cleans and transforms it, stores it in SQLite, and runs SQL queries to produce analytical outputs. |
| Analytics and Machine Learning | Performs exploratory data analysis and predictive modelling on the Titanic dataset, evaluates models, and offers an interactive prediction interface. |
| AI Support Assistant | Retrieves relevant information from policy documents and generates answers to user questions through a RAG workflow. |
| Streamlit Interface | Central UI providing the dashboard, analytics and ML features, and the AI Support Assistant chat. |

The project shows how components developed independently can be integrated into a single application.

# 2. Problem Statement

Data-driven applications require several stages of processing before useful results reach users. Raw information must be collected, cleaned, stored, analysed and transformed into meaningful outputs. Predictive models need appropriate preprocessing and evaluation, while AI-based support systems need a mechanism for retrieving relevant information and producing context-aware responses. Developing these components separately makes the overall workflow difficult to demonstrate and maintain.

The objective is to build an integrated application that demonstrates:

- Automated extraction of structured information from web pages.
- Data cleaning, transformation, and database storage.
- SQL-based analysis and validation.
- Exploratory data analysis and machine learning.
- Retrieval of relevant information from policy documents.
- API-based communication between backend and frontend.
- A unified, interactive user interface.

# 3. Project Objectives

1. Extract structured information from a public website using Python.
2. Clean and transform raw data into an analysis-ready format.
3. Store structured data in a relational SQLite database.
4. Perform SQL queries and export analytical results.
5. Use Pandas for data manipulation and validation.
6. Explore datasets using descriptive statistics and visualisations.

7 Train and evaluate machine learning models.

8 Save a trained model for subsequent inference.

9 Implement a document-based AI Support Assistant using RAG.

10 Expose chatbot functionality through a FastAPI backend.

11 Use Uvicorn to serve the API locally.

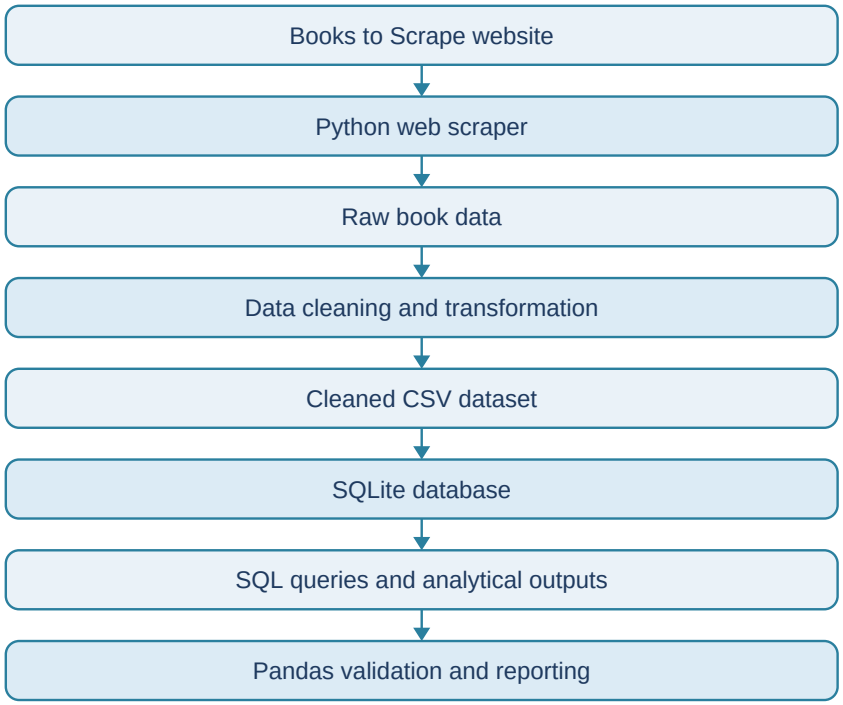
12 Develop a Streamlit interface that integrates the project modules.

13 Organise the application into reusable, maintainable components.

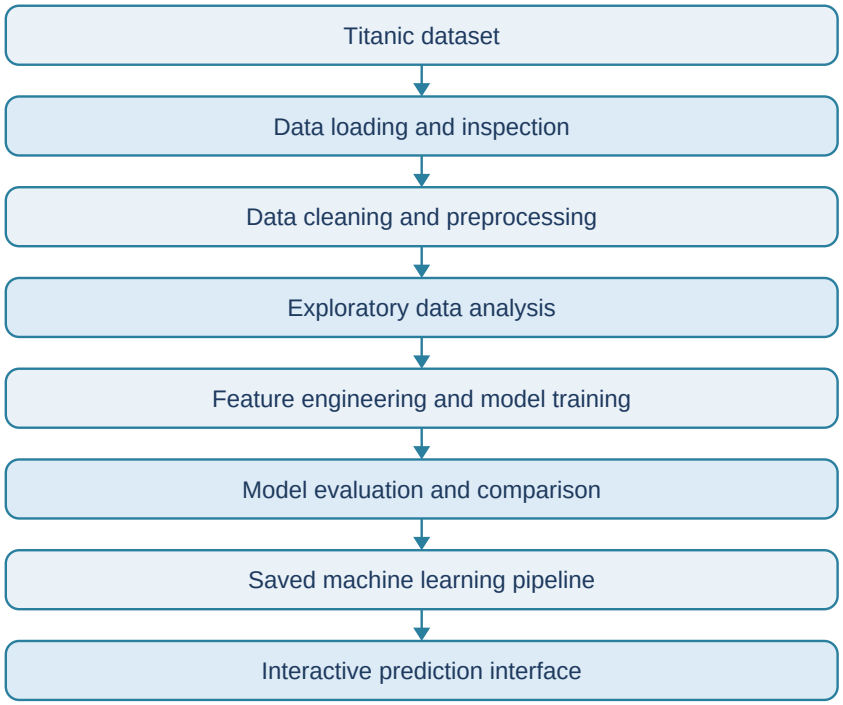
# 4. System Architecture

The application consists of three functional pipelines brought together by the Streamlit interface.

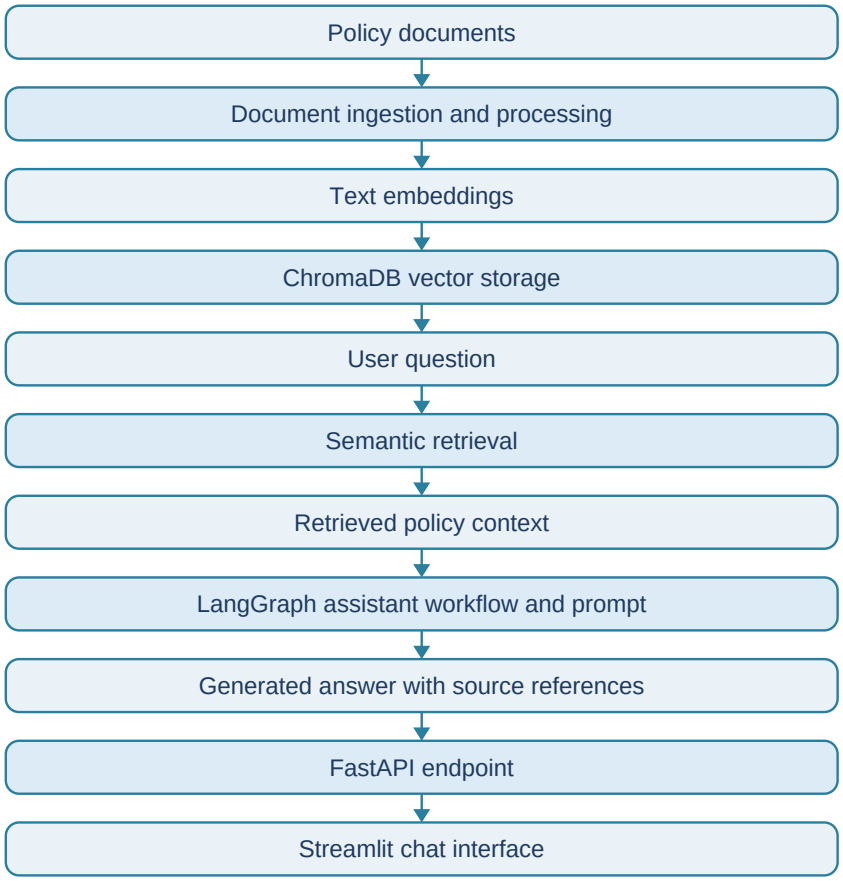
## 4.1 Data Engineering Architecture



## 4.2 Analytics and Machine Learning Architecture



## 4.3 AI Support Assistant Architecture




The exact internal execution order of the LangGraph nodes and the embedding configuration should be confirmed against `graph.py`, `ingestion.py`, and `retrieval.py`.

# 5. Project Directory Structure

The main project directory is `C:\Users\Dell\PycharmProjects\CapstoneProject`. The structure below is representative of the files inspected; some files and details may vary as the project evolves.

```
CapstoneProject/
|
+-- analytics/
|   +-- titanic.csv
|   +-- titanic_cleaned.csv
|   +-- best_titanic_model.joblib
|   +-- cleaning.py
|   +-- data_loading.py
|   +-- profiling.py
|   +-- univariate_analysis.py
|   +-- bivariate_analysis.py
|   +-- multivariate_analysis.py
|   +-- modeling.py
|   +-- random_forest_tuning.py
|   +-- final_model_comparison.py
|   +-- fare_regression.py
|
+-- data_pipeline/
|   +-- scraper.py
|   +-- cleaning.py
|   +-- database.py
|   +-- sqlite.py
|   +-- datapipeline_sqlite.py
|   +-- pandas_validation.py
|   +-- books.db
|   +-- raw_books.csv
|   +-- cleaned_books.csv
|   +-- query output CSV files
|
+-- support_assistant/
|   +-- documents/
|   +-- chroma_db/
|   +-- api.py
|   +-- graph.py
|   +-- ingestion.py
|   +-- prompt.py
|   +-- retrieval.py
|   +-- schemas.py
|
+-- UI/
|   +-- app.py
|   +-- pages/
|       +-- dashboard.py
|       +-- analytics.py
|       +-- support_assistant.py
|
+-- Dockerfile
+-- requirements.txt
+-- pyproject.toml
+-- uv.lock
+-- README.md
```

# 6. Module One: Web Scraping and Data Engineering

## 6.1 Introduction to Web Scraping

Web scraping is the automated extraction of information from web pages into structured data. The source website is *Books to Scrape*, a site commonly used for learning and testing scraping techniques. This module collects book-related information, processes it, and makes it available for database operations and analysis. The main script is `data_pipeline/scraper.py`.

## 6.2 Extracting Data from Web Pages

The scraper uses **Requests** to retrieve page HTML and **BeautifulSoup** to parse it and locate elements via tags, classes, and attributes. A typical workflow is:

1 Send an HTTP request to the website.

2 Check whether the response is successful.

3 Parse the HTML response.

4 Locate book entries on each page.

5 Extract available fields.

6 Move through the required pages.

7 Collect the extracted records.

8 Save the records into a structured file.

The earlier workflow collected five pages containing approximately 100 book records. Typical attributes include title, price, availability, rating, and category, but the precise field list *should be verified in scraper.py*.

## 6.3 Raw Data Storage

Extracted records are stored in `data_pipeline/raw_books.csv`. CSV gives a simple tabular representation. The raw dataset is an intermediate artefact between extraction and cleaning; keeping it separate from the cleaned data makes it easier to inspect extraction problems and reproduce later stages.

## 6.4 Data Cleaning and Transformation

Raw web data can contain inconsistent formatting, unwanted characters, duplicates, missing values, and numbers stored as text. The cleaning module is `data_pipeline/cleaning.py`. The earlier implementation converted prices between GBP and INR using a rate of **Rs. 105.50 per GBP**. This is a configured project assumption, not a live exchange rate.

1 Load the raw CSV.

2 Inspect column names and data types.

3 Standardise textual fields.

4 Convert numeric fields to appropriate types.

5 Handle missing or invalid values per the cleaning logic.

6 Apply the currency conversion.

7 Validate the transformed records.

8 Export the cleaned dataset.

The cleaned data is stored in `data_pipeline/cleaned_books.csv`.

## 6.5 SQLite Database Integration

SQLite is a lightweight relational database stored in a single file and needs no separate server. The project database is `data_pipeline/books.db`, managed by `database.py`, `sqlite.py` and `datapipeline_sqlite.py`.

The earlier workflow established a normalised design with **books** and **categories** tables. Normalisation avoids repeating category information in every book record by keeping it in a separate table referenced by the book table. Conceptually: **Categories (one) to Books (many)**. Exact column names, keys, and constraints should be taken from the SQL schema.

## 6.6 SQL Query Execution

Once data is loaded into SQLite, SQL retrieves and summarises information. The project contains six query outputs saved as `query1_output.csv` through `query6_output.csv`, demonstrating reusable analytical results. Typical operations include filtering, sorting, grouping, aggregation, joins, and summary statistics. The business question answered by each query should be documented from `datapipeline_sqlite.py` rather than inferred from filenames.

## 6.7 Pandas Validation

Validation is implemented in `data_pipeline/pandas_validation.py`. The earlier workflow used `pd.read_sql` and merge-based comparison between database query results and independently processed data; a result of `Equivalent: True` was observed. This is evidence that the tested outputs matched under the comparison performed, not a guarantee that every operation or row was independently verified.

## 6.8 Module Outputs

- Raw scraped data.
- Cleaned CSV data.
- A populated SQLite database.
- SQL query result files.
- Pandas-based validation results.

# 7. Module Two: Data Analytics and Machine Learning

## 7.1 Overview

This module shows how a dataset can be examined, cleaned, visualised, and used to train predictive models. It uses the Titanic dataset with separate scripts for loading, cleaning, profiling, exploratory analysis, model training, comparison, and regression.

| Artefact | Path |
|---|---|
| Primary dataset | analytics/titanic.csv |
| Cleaned dataset | analytics/titanic_cleaned.csv |
| Saved model | analytics/best_titanic_model.joblib |

## 7.2 Data Loading and Inspection

The dataset is read into a Pandas DataFrame and contains **891 rows and 15 columns**. Inspection involves examining dimensions, viewing sample records, identifying column names, checking data types, computing descriptive statistics, identifying missing values, and examining duplicates where relevant.

## 7.3 Data Cleaning

Cleaning is associated with `analytics/cleaning.py` and can include handling missing values, converting data types, removing or retaining columns, and preparing categorical and numeric features. The treatment of each feature must be documented from the script; it should not be assumed that every missing value was removed or every categorical variable encoded identically.

## 7.4 Exploratory Data Analysis

EDA helps understand distributions, relationships, patterns, and data-quality problems. Scripts: `profiling.py`, `univariate_analysis.py`, `bivariate_analysis.py`, `multivariate_analysis.py`.

| Type | Purpose | Examples |
|---|---|---|
| Univariate | Examines one variable at a time | Age distribution, passenger class counts, fare distribution, survival counts |
| Bivariate | Relationship between two variables | Survival vs passenger class, sex, or age |
| Multivariate | Relationships among several variables | Correlation analysis and related visualisations |

The Streamlit analytics page presents selected charts and tables. Observed associations do not by themselves establish causal relationships.

## 7.5 Classification: Titanic Survival Prediction

The classification component predicts whether a passenger survived. Scripts include `modeling.py`, `python_modeling.py`, `random_forest_tuning.py`, `final_model_comparison.py` and `save_best_pipeline.py`. The workflow:

1 Select the target variable.

2 Prepare input features.

3 Split data into training and testing sets.

4 Preprocess features as required.

5 Train candidate classification models.

6 Evaluate predictions.

7 Tune hyperparameters.

8 Compare model performance.

9 Save the selected model pipeline.

### Recorded model comparison

| Model | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.7809 | 0.7544 | 0.6324 | 0.6880 | 0.8265 |
| Decision Tree | 0.7697 | 0.7143 | 0.6618 | 0.6870 | 0.7412 |
| Random Forest | 0.7978 | 0.7759 | 0.6618 | 0.7143 | 0.8211 |

These values were recorded in the project interface and prior testing. Consult the evaluation scripts to confirm the split, preprocessing, and metric calculations before using them in a formal report.

## 7.6 Random Forest Hyperparameter Tuning

The tuned Random Forest uses a maximum tree depth of five and square-root feature selection.

| Metric | Recorded result |
|---|---|
| Test accuracy | 0.8034 |
| Cross-validation F1-score | 0.7436 |
| Out-of-bag score | 0.8073 |

Cross-validation estimates performance across multiple data splits. The out-of-bag score is a Random Forest-specific estimate using training samples not selected in individual bootstrap samples. These metrics measure different aspects of performance and are not interchangeable.

## 7.7 Model Serialisation

The selected pipeline is saved as `analytics/best_titanic_model.joblib`. Joblib lets the trained pipeline be loaded later without retraining at each dashboard start. When the artefact is a complete pipeline, saving preprocessing together with the estimator ensures new inputs receive the same transformations used in training. Contents should be verified in `save_best_pipeline.py`.

## 7.8 Fare Regression

The component in `analytics/fare_regression.py` evaluates a model predicting passenger fare.

| Metric | Value | Meaning |
|---|---|---|
| MAE | 21.0986 | Average absolute prediction error |
| RMSE | 41.7021 | Penalises larger errors more heavily |
| $R^2$ | 0.3482 | Share of target variance explained vs a mean baseline |

| Metric | Value | Meaning |
|---|---|---|
| Adjusted $R^2$ | 0.3173 | $R^2$ adjusted for number of predictors |

The recorded $R^2$ indicates that the model explains a limited portion of fare variability. This should be reported honestly rather than presenting the model as highly accurate. A fare-regression residual visualisation is also included.

## 7.9 Interactive Analytics Interface

Implemented in `UI/pages/analytics.py`, the page loads the dataset and saved model and includes a dataset overview, data preview, missing-value inspection, exploratory charts, correlation analysis, classification comparisons, tuned Random Forest results, fare-regression metrics, and an interactive survival prediction form.

| # | Prediction form input |
|---|---|
| 1 | Passenger class |
| 2 | Sex |
| 3 | Age |
| 4 | Number of siblings or spouses aboard |
| 5 | Number of parents or children aboard |
| 6 | Fare |
| 7 | Embarkation port |

The application passes the inputs to the saved model and displays the prediction and probability when available. Feature names, ordering, encoding, and value ranges must remain compatible with the saved pipeline.

# 8. Module Three: AI Support Assistant

## 8.1 Purpose

The assistant is a document-based question-answering system over a collection of Zepto-style policy documents. Unlike a conventional chatbot, a retrieval-augmented generation system first retrieves relevant information and uses it as context. It covers delivery, returns, tracking, cancellation, membership, damaged or missing items, gift cards, and customer support.

> The policy documents are illustrative capstone data and must not be treated as verified statements of current real-world Zepto policies.

## 8.2 Policy Documents

Documents reside in `support_assistant/documents/`. Answer quality depends on document accuracy, consistency and completeness, as well as retrieval and prompting logic.

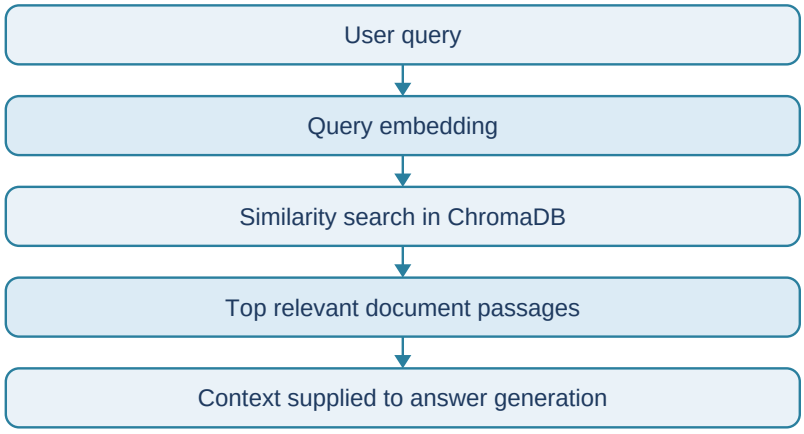
| Document | Policy topic |
|---|---|
| doc_01 | Delivery |
| doc_02 | Returns |
| doc_03 | Account tiers and membership |
| doc_04 | Order tracking |
| doc_05 | Order cancellation |
| doc_06 | Damaged, spoiled, or missing items |
| doc_07 | Gift cards |
| doc_08 | Customer support |

## 8.3 Document Ingestion

Ingestion is in `support_assistant/ingestion.py`. The application reads the documents, extracts text, associates it with identifiers and metadata, converts it to vector embeddings, and stores vectors and text in a vector database. The project uses ChromaDB and the sentence-transformer model **all-MiniLM-L6-v2**. Chunking strategy, embedding initialisation and ingestion trigger should be confirmed in `ingestion.py`.

## 8.4 Embeddings and Semantic Search

An embedding represents text as a numeric vector; semantically related passages have similar vectors even without shared words. The retrieval module is `support_assistant/retrieval.py`, using the ChromaDB collection `zepto_policies` persisted under `support_assistant/chroma_db/`. The recorded configuration returns the **top three** results, each with a document ID and similarity-distance value. For the damaged-item query, `doc_06` ranked first.




### Understanding retrieval distance

A lower distance generally indicates a closer match under the configured metric. It is not a percentage probability that the result is correct; its interpretation depends on the distance metric.

## 8.5 LangGraph Workflow

The workflow is associated with `support_assistant/graph.py`. LangGraph organises an application into connected processing nodes. Conceptually, the workflow receives a question, works with retrieved context, prepares an answer-generation request, and returns a response. The actual nodes, state schema, conditional transitions and model provider must be documented from `graph.py`; use of LangGraph alone does not establish agents, tools, or multiple autonomous reasoning steps.

## 8.6 Prompt Engineering

The prompt module is `support_assistant/prompt.py`. A well-designed policy Q&A; prompt should instruct the model to:

- Use the retrieved policy context.
- Answer the user's specific question.
- Avoid inventing policy details.
- Acknowledge when the context lacks enough information.
- Distinguish relevant policy statements from unrelated retrieved content.

These are recommended grounding requirements; the implemented prompt must be checked in `prompt.py`. Retrieving the correct document does not guarantee the final answer uses it correctly.

## 8.7 Request and Response Schemas

Defined in `support_assistant/schemas.py`, schemas specify the structure of API inputs and outputs. The Streamlit client sends a JSON object with a `query` field and reads `answer`, `sources` and `confidence` from the response. Validation rules should be confirmed in `schemas.py`.

## 8.8 FastAPI Backend

The API module is `support_assistant/api.py`. The frontend calls `POST /ask` with a JSON body, and the backend processes it through the assistant workflow and returns a structured response.

### Representative request

```
{
  "query": "How can I track my order?"
}
```

### Representative response (illustrative only)

```
{
  "answer": "Answer generated from retrieved policy context",
  "sources": ["doc_04", "doc_01"],
  "confidence": 0.90
}
```

> The confidence value is displayed by the interface, but its meaning depends on how the backend computes it. It should not be described as a calibrated probability unless the implementation supports that interpretation.

## 8.9 Uvicorn and the API Server

FastAPI defines the application and endpoints; Uvicorn, an ASGI server, serves it over HTTP. From the project root:

```
python -m uvicorn support_assistant.api:app --host 127.0.0.1 --port 8000
```

| Part | Meaning |
|---|---|
| python -m uvicorn | Runs Uvicorn using the active Python environment |
| support_assistant.api | Python module containing the API |
| :app | The FastAPI application object |
| --host 127.0.0.1 | Binds to the local loopback interface |
| --port 8000 | Uses port 8000 for HTTP requests |

Interactive API documentation is available at `http://127.0.0.1:8000/docs`. Because the server binds to 127.0.0.1, it is reachable only from the same computer.

## 8.10 Streamlit-to-FastAPI Communication

The frontend (`UI/pages/support_assistant.py`) sends requests to `http://127.0.0.1:8000/ask`.

1. The user enters a question in Streamlit.
2. The frontend sends an HTTP POST request.
3. FastAPI validates and processes the request.
4. The workflow retrieves relevant policy context.
5. The answer-generation component produces a response.
6. The backend returns a JSON response.
7. Streamlit displays the answer, sources, and confidence value.

The frontend uses a request timeout and handles connection failures, timeouts, HTTP errors, and unexpected exceptions. The separation keeps retrieval and generation logic out of the UI.

## 8.11 Testing the RAG Assistant

| Test | Question | Observed result | Assessment |
|---|---|---|---|
| 1 | How long do I have to report a damaged grocery item? | 24-hour reporting window; refers to Report an Issue; top document doc_06. | Relevant and aligned with policy text. |
| 2 | How can I track my order? | Explains Track Order screen, rider tracking, delivery-time updates; sources doc_04, doc_06, doc_01. | Expected document retrieved; others may be less relevant. |
| 3 | Can I cancel an order after it is packed? | Distinguishes cancellation before and after packing; automatic cancellation and refunds; sources doc_05, doc_02, doc_06. | Expected document retrieved; extra results may add noise. |
| 4 | How much does Zepto Pass cost? | Describes Basic, Zepto Pass, and Zepto Pass+ with prices and benefits; sources doc_03, doc_01, doc_07. | Expected membership document retrieved. |

**Testing limitation:** these tests show relevant answers for several questions but do not establish perfect retrieval precision or eliminate generic-answer failures. Further testing should include questions with no answer in the knowledge base, ambiguous queries, paraphrased questions, and queries that may retrieve overlapping policies.

# 9. Module Four: Streamlit User Interface

## 9.1 Overview

Streamlit builds the interactive web interface in Python. The entry point is `UI/app.py`, which uses page navigation to present the three principal pages.

## 9.2 Main Dashboard

`UI/pages/dashboard.py` gives a high-level introduction: project overview, module descriptions, and the technology stack. It acts as a central starting point before users move to analytics or the assistant.

## 9.3 Analytics and ML Page

`UI/pages/analytics.py` loads the Titanic dataset and serialised model, presents summaries and visualisations, and provides the interactive prediction interface.

## 9.4 AI Support Assistant Page

`UI/pages/support_assistant.py` includes:

- A chat interface.
- Suggested policy questions.
- Conversation history kept in Streamlit session state.
- HTTP requests to the FastAPI backend.
- Answer display.
- Retrieved source display.
- Confidence display.
- Error handling for failed API requests.

## 9.5 Styling and Navigation

The main UI file configures the title, page icon, wide layout, and initial sidebar state, and contains custom CSS for the background, sidebar, metric cards, buttons, dataframes, and chat messages. The sidebar shows project information and the technology stack. Navigation uses `st.navigation()` and `st.Page()`; the selected page runs via `pg.run()`.

# 10. Running the Complete Application

Two processes must run during normal local use.

## 10.1 Start the FastAPI Backend

```
cd C:\Users\Dell\PycharmProjects\CapstoneProject
.\.venv\Scripts\Activate.ps1
python -m uvicorn support_assistant.api:app --host 127.0.0.1 --port 8000
```

Keep this terminal open and verify the backend at `http://127.0.0.1:8000/docs`.

### 10.2 Start the Streamlit Frontend

```
cd C:\Users\Dell\PycharmProjects\CapstoneProject
.\.venv\Scripts\Activate.ps1
python -m streamlit run UI/app.py
```

Keep this terminal open as well. The interface normally opens at `http://localhost:8501`; otherwise enter the address in a browser.

### 10.3 Why Two Terminals Are Needed

| Process | Server | Port |
|---|---|---|
| FastAPI backend | Uvicorn | 8000 |
| Streamlit frontend | Streamlit | 8501 |

The frontend sends requests to the backend whenever a chatbot question is submitted. If the backend is stopped, the chatbot may report a connection error while the dashboard remains accessible. The analytics page loads its dataset and model locally and does not necessarily depend on the backend.

## 11. Database Architecture: SQLite and ChromaDB

| Aspect | SQLite | ChromaDB |
|---|---|---|
| Used by | Data-engineering pipeline | AI Support Assistant |
| Location | data_pipeline/books.db | support_assistant/chroma_db/ |
| Stores | Structured book records in relational tables | Vector representations of policy text |
| Query style | SQL | Semantic similarity retrieval |
| Best for | Structured records, relational queries, tabular outputs | Retrieving semantically relevant passages |

The presence of both databases does not mean they exchange data; integration depends on the code that accesses each one. The SQLite scripts are not normally needed merely to start the FastAPI assistant if it already initialises its ChromaDB retrieval system.

# 12. Error Handling and Troubleshooting

## 12.1 FastAPI Backend Does Not Start

- The virtual environment is activated.
- Required dependencies are installed.
- The command is run from the project root.
- The module path support_assistant.api:app is correct.
- Port 8000 is not already occupied.

## 12.2 Streamlit Cannot Connect to the Assistant

- Uvicorn is still running.
- The API is reachable at http://127.0.0.1:8000/docs.
- The frontend uses the correct endpoint, http://127.0.0.1:8000/ask.
- Backend and frontend run in the intended Python environment.

## 12.3 Chatbot Returns Generic or Irrelevant Answers

Possible causes: the retriever returns irrelevant documents; correct documents are not indexed; the retrieval configuration is unsuitable; retrieved text is not passed correctly into the prompt; the prompt encourages generic responses; or the model or fallback logic ignores missing context. Suggested debugging sequence:

1. Run `python -m support_assistant.retrieval`.
2. Enter the question that produces a generic answer.
3. Inspect the top retrieved documents.
4. Check whether the expected document is among the results.
5. Inspect how `graph.py` receives and uses retrieved context.
6. Review the instructions in `prompt.py`.
7. Test the same question through the `/ask` endpoint.

Fix the stage where the failure occurs rather than modifying the UI without evidence.

## 12.4 Model or Dataset Loading Failure

Check that the CSV and `.joblib` files exist at the paths expected by `UI/pages/analytics.py`. A model saved with a different feature schema may fail during prediction even if the file loads, so input compatibility should be tested separately.

# 13. Technologies Used

| Technology | Role in the project |
|---|---|
| Python | Main programming language |
| Requests | HTTP requests for web scraping |
| BeautifulSoup | HTML parsing and extraction |
| Pandas | Data cleaning, transformation, and analysis |
| SQLite | Relational database storage |
| SQL | Querying and aggregation |
| Scikit-learn | Machine learning and model evaluation |
| Joblib | Saving and loading model artefacts |
| Streamlit | Interactive dashboard and chat UI |
| FastAPI | HTTP API backend |
| Uvicorn | ASGI server for FastAPI |
| Sentence Transformers | Text embeddings for semantic retrieval |
| ChromaDB | Vector database for policy retrieval |
| LangGraph | Workflow orchestration for the assistant |
| Docker | Containerisation support |

Versions and the full dependency list should be taken from `requirements.txt`, `pyproject.toml` or `uv.lock`.

# 14. Project Outcomes

The project integrates several practical software and data-science capabilities. The data-engineering module produces cleaned book data, SQLite storage, SQL query outputs, and validation results. The analytics module presents exploratory analysis, classification comparisons, a tuned Random Forest, and a separate fare-regression evaluation. The AI Support Assistant retrieves policy documents and returns source-associated responses through an API, and Streamlit provides a unified interface.

Initial testing confirmed relevant retrieval for tracking, cancellation, membership, and damaged-item questions. However, broader validation of generic-answer behaviour, source relevance, prediction functionality, and end-to-end reliability is still required before any component is considered production-ready.

# 15. Limitations and Future Enhancements

## 15.1 Current Limitations

1. The AI assistant can return generic answers for some questions.
2. The retriever may return additional documents that are not essential to a query.
3. The displayed confidence score requires further examination to establish its meaning.
4. Model results depend on the dataset, preprocessing, evaluation split, and selected features.

5 The application uses local server addresses for frontend-to-backend communication.

6 The SQLite data pipeline and ChromaDB knowledge base are separate storage systems.

7 The project is not established as a production deployment with authentication, monitoring, and operational safeguards.

### 15.2 Future Enhancements

- Improve document chunking and retrieval relevance.
- Introduce a retrieval threshold and better handling of unsupported questions.
- Improve prompt grounding and reduce hallucinations.
- Evaluate retrieval with a defined question-and-document test set.
- Add automated tests for API endpoints and prediction inputs.
- Improve validation and error messages in the UI.
- Record application logs for troubleshooting.
- Use configuration files or environment variables for deployment settings.
- Containerise and test the application with Docker.
- Document the database schema and SQL query objectives.
- Add a reproducible model evaluation report.
- Perform security, performance, and deployment testing before external use.

## 16. Conclusion

The AI/ML Capstone Project demonstrates an end-to-end workflow combining data engineering, analytics, machine learning, semantic retrieval, API development, and interactive UI development. The web-scraping module extracts book information and structures it; the SQLite pipeline provides relational storage and SQL analysis; the analytics module explores the Titanic dataset, evaluates models, and serves a saved model for interactive prediction.

The AI Support Assistant combines policy documents, text embeddings, ChromaDB retrieval, a graph-based workflow, and a FastAPI endpoint. Uvicorn serves the API locally, while Streamlit provides the dashboard and chatbot interface.

The project also illustrates an engineering principle: a working interface is only one part of a complete system. Reliable outputs require correct data processing, appropriate model evaluation, relevant retrieval, grounded answer generation, and systematic end-to-end testing.

# Appendix A: Important Project Files

| File | Purpose |
|---|---|
| data_pipeline/scraper.py | Web-data extraction |
| data_pipeline/cleaning.py | Data cleaning and transformation |
| data_pipeline/database.py | Database-related operations |
| data_pipeline/sqlite.py | SQLite-related operations |
| data_pipeline/datapipeline_sqlite.py | Data-pipeline and SQL workflow |
| data_pipeline/pandas_validation.py | Data and query-result validation |
| analytics/data_loading.py | Analytics data loading |
| analytics/cleaning.py | Analytics data cleaning |
| analytics/modeling.py | Model development |
| analytics/random_forest_tuning.py | Random Forest tuning |
| analytics/final_model_comparison.py | Model comparison |
| analytics/save_best_pipeline.py | Model artefact creation |
| analytics/fare_regression.py | Regression analysis |
| support_assistant/ingestion.py | Policy-document ingestion |
| support_assistant/retrieval.py | Semantic retrieval |
| support_assistant/graph.py | Assistant workflow |
| support_assistant/prompt.py | Answer-generation instructions |
| support_assistant/schemas.py | API data schemas |
| support_assistant/api.py | FastAPI application |
| UI/app.py | Main Streamlit entry point |
| UI/pages/dashboard.py | Project dashboard |
| UI/pages/analytics.py | Analytics and ML interface |
| UI/pages/support_assistant.py | AI assistant interface |

# Appendix B: Demonstration Sequence

For a project demonstration or viva, the following order gives a coherent explanation:

1. Introduce the overall project and its three modules.
2. Demonstrate the scraper and explain raw versus cleaned data.
3. Show the SQLite database and SQL query outputs.
4. Explain the Pandas validation process.
5. Open the Analytics & ML page and demonstrate EDA.
6. Explain the classification metrics and regression results.
7. Submit an example prediction through the UI.

8 Explain policy-document ingestion and ChromaDB retrieval.

9 Demonstrate a policy question in the AI Support Assistant.

10 Expand the retrieved sources and explain how they support the answer.

11 Show the FastAPI documentation at /docs.

12 Explain how Uvicorn serves the backend and how Streamlit communicates with it.

13 Conclude with limitations and planned improvements.